

借贷协议期末考试

SimpleLendingPool · Foundry 学习代码配套

课程：智能合约安全审计	总分：100 分	建议时长：45 分钟
题型：单选 5 题 + 计算 3 题 + 简答 2 题	参考代码： SimpleLendingPool.sol	允许：草稿纸 / 代码参考

第一部分 单选题（每题 6 分，共 30 分）

说明：每题只有一个正确答案。

1

(6
分)

存款人在协议中通过哪种机制获得利息收益？

- A 协议每天将利息直接转入存款人钱包
- B `exchangeRate` 随时间上升，持有相同 `IToken` 可兑换更多底层资产
- C `IToken` 的 `totalSupply` 减少，使每份额价值上升
- D 借款人直接调用 `payInterest()` 向存款人地址付款

参考答案

正确答案：B

解析：存款人持有的 `IToken` 数量不变，但 $\text{exchangeRate} = (\text{cash} + \text{totalBorrows}) / \text{ITokenSupply}$ 。随着借款人还息，`totalBorrows` 增大 → `exchangeRate` 上升 → 取款时兑换更多 USDC。
`IToken` 总量不减少，利息也不主动 `push`，所以 A/C/D 均错误。

2

(6
分)

`borrowIndex` 的核心设计目的是什么？

- A 记录每个用户的借款次数，方便限流
- B 存储当前最优利率，供利率模型查询
- C 以 $O(1)$ 复杂度实现全局利息更新，无需遍历所有用户
- D 防止重入攻击，作为互斥锁使用

参考答案

正确答案：C

解析：借款人借款时记录 `interestIndex` 快照，任意时刻 $\text{actualDebt} = \text{principal} \times \text{borrowIndex_now} / \text{RAY}$ 。

协议只需更新一个全局变量，无需遍历账户，是 Compound V2 最重要的设计之一。

3 (6分) Kink 利率模型中，利用率超过 `OPTIMAL_UTILIZATION`（80%）后，利率曲线为何变得很陡？

- A 激励新存款人进入并威慑借款人，防止池子流动性耗尽
- B 这是 `Solidity` 整数运算的自然结果，并非刻意设计
- C 高利用率时协议需要积累更多 `reserve`，故意抬高利率
- D 防止套利机器人在低利率时大量借款

参考答案

正确答案：A

解析：当利用率极高时，存款人随时可能无法取款（流动性危机）。

高利率同时发挥两个作用：吸引外部存款进入 + 迫使借款人提前还款，使利用率自动回落。

4 (6分) 以下关于清算（`liquidation`）的说法，哪个是正确的？

- A 健康因子 < 1 时，协议自动执行清算，无需外部调用
- B 清算人获得的抵押物价值严格等于其代还的债务价值
- C 被清算用户必须提前授权清算人操作自己的账户
- D 清算人代还部分债务后，获得等值抵押物加上 5% 的奖励（`bonus`）

参考答案

正确答案：D

A 错：协议不会自动执行，需要外部账户（清算机器人）主动调用 `liquidate()`。

B 错：清算人获得的是「等值 + 5% `bonus`」，否则没有利润动机。

C 错：`liquidate()` 是无需授权的开放函数，任何人都可清算 `HF < 1` 的账户。

5 (6分) 为什么所有写入操作（`borrow`、`repay`、`withdraw` 等）执行前必须先调用 `accrueInterest()`？

- A 满足 `Solidity` 编译器的要求，否则无法通过编译

- B 保证 borrowIndex 是最新值，使后续利息和健康因子计算正确
- C 防止 gas 浪费，将多次操作的 gas 合并一次
- D 防止重入，accrueInterest() 内部有 reentrancy guard

参考答案

正确答案：B

解析：accrueInterest() 根据经过的秒数更新 borrowIndex。

若先执行操作再 accrue，则用过时的 index 计算债务和健康因子，产生错误结果。

这也是测试中「skip 时间后需显式调用 accrueInterest()」的根本原因。

第二部分 计算题（每题 12 分，共 36 分）

说明：请写出完整计算过程，只写答案不得分。

6

(12
分)

已知：BASE_RATE = 2%，SLOPE1 = 10%，OPTIMAL_UTILIZATION = 80%。
当前资金池中有 1000 USDC 存款，已借出 600 USDC（池中剩余 400 USDC）。

- ① 计算当前资金利用率（Utilization）
- ② 计算当前借款年化利率（APR）

① 利用率 =

② 借款 APR =

参考答案

① 利用率：totalBorrows / (cash + totalBorrows) = 600 / (400 + 600) = 600 / 1000 = 60.0%

② 利用率 60% < OPTIMAL (80%)，使用低区间公式：
rate = BASE_RATE + SLOPE1 × (utilization / OPTIMAL)
= 2% + 10% × (0.60 / 0.80)
= 2% + 10% × 0.75 = 2% + 7.5% = 9.5%

7

(12
分)

已知：Alice 在 borrowIndex = 1.0 × RAY 时借出 1000 USDC。
1 年后 borrowIndex 更新为 1.08 × RAY（年化利率约 8%）。

- ① 写出 Alice 借款时存入 BorrowSnapshot 的 normalizedPrincipal 值
- ② 计算 1 年后 Alice 的实际债务（actualDebt）

③ 本次借款产生的利息是多少？

① `normalizedPrincipal` =

② `actualDebt` =

③ 利息 =

参考答案

① $\text{normalizedPrincipal} = \text{actual} / \text{borrowIndex} \times \text{RAY} = 1000 / 1.0 \times \text{RAY} = 1000 \times \text{RAY}$ （即 $1000\text{e}27$ ）

② $\text{actualDebt} = \text{normalizedPrincipal} \times \text{borrowIndex_now} / \text{RAY}$
 $= (1000 \times \text{RAY}) \times (1.08 \times \text{RAY}) / \text{RAY} = 1000 \times 1.08 = 1080 \text{ USDC}$

③ 利息 = $\text{actualDebt} - \text{原始借款} = 1080 - 1000 = 80 \text{ USDC}$

8

(12
分)

已知：Bob 存入 50 WETH 作为抵押物。

合约参数：`COLLATERAL_PRICE` = 2（1 WETH = 2 USDC），`LTV` = 75%，`LIQUIDATION_THRESHOLD` = 80%。

① Bob 最多可以借出多少 USDC（`maxBorrowable`）？

② 若 Bob 借了 70 USDC，当前健康因子（HF）是多少？保留两位小数。

③ 健康因子降到多少时 Bob 可被清算？写出清算触发的不等式。

① `maxBorrowable` =

② `healthFactor` =

③ 清算触发条件：

参考答案

① $\text{maxBorrowable} = \text{抵押物价值} \times \text{LTV} = (50 \times 2) \times 75\% = 100 \times 75\% = 75 \text{ USDC}$

② $\text{HF} = (\text{抵押价值} \times \text{LIQUIDATION_THRESHOLD}) / \text{debt}$
 $= (50 \times 2 \times 80\%) / 70 = 80 / 70 \approx 1.14$

③ 清算条件： $\text{HF} < 1.0$ ，即 $\text{debt} > \text{collateralValue} \times \text{LIQUIDATION_THRESHOLD}$
等价于： $\text{debt} > 50 \times 2 \times 80\% = 80 \text{ USDC}$

第三部分 简答题（每题 17 分，共 34 分）

说明：结合合约代码作答，需说明原因和影响。

9
(17
分)

下面是 `accrueInterest()` 中的一段代码，请回答两个问题：

```
行 A: uint256 interestAccumulated = getTotalBorrows();  
行 B: borrowIndex = newBorrowIndex;  
行 C: uint256 newTotalBorrows = getTotalBorrows();  
emit AccrueInterest(..., newTotalBorrows - interestAccumulated, ...);
```

- ① 行 A 和行 C 的 `getTotalBorrows()` 使用的 `borrowIndex` 一样吗？`emit` 参数计算的是什么？
- ② 若在 `skip(365 days)` 后直接调用 `view` 函数读取债务，而不触发 `accrueInterest()`，会发生什么？

（在此处作答）

参考答案

- ① 不一样。行 A 在 `borrowIndex` 更新前调用，使用旧 `index`；

行 C 在更新后调用，使用新 `index`。

两者之差 = 本次新增利息，`emit` 参数语义正确。

但代码可读性差，容易被误认为是同一时刻的两次调用。

- ② `borrowIndex` 是状态变量，只在 `accrueInterest()` 被调用时更新。

`view` 函数不会触发 `accrue`，所以读到的是过去时刻的旧 `index`。

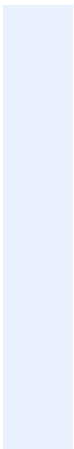
导致债务/健康因子被低估（利息未计入），这正是测试中多个 `case` 失败的根本原因。

10
(17
分)

从安全审计角度，指出 `SimpleLendingPool` 合约中至少 3 个潜在风险点。

每个风险点需说明：① 风险描述 ② 触发场景 ③ 修复思路（一句话即可）。

（在此处作答）



参考答案

参考答案（任意 3 点均可得满分）：

- ① 价格 Oracle 硬编码
风险：COLLATERAL_PRICE 为常量，无法反映市场波动。
场景：抵押物价格暴跌时，协议仍按高价计算抵押率，导致大量账户资不抵债。
修复：接入 Chainlink，并使用 TWAP 防止闪贷操纵价格。
- ② 缺少 Reentrancy Guard
风险：borrow()/withdraw() 在 transfer 前更新状态，但若 token 有 hook，
场景：ERC777 或有回调的 token 可重入，在状态写入前反复调用。
修复：添加 nonReentrant modifier，或严格遵循 Checks-Effects-Interactions 模式。
- ③ 清算无 close factor 限制
风险：清算人可一次性还清全部债务，对被清算人惩罚过重。
场景：轻微抵押不足时，被一次清算至零，协议可能同时产生坏账。
修复：引入 close factor（如 50%），每次清算最多还清一半债务。
- ④ 利息计算使用线性近似而非精确复利
风险：长时间不 accrue 时，利息低估，存款人收益被侵蚀。
修复：使用指数函数精确复利，或限制 accrue 最大间隔（如 Compound 做法）。
- ⑤ 无最大借款比例检查（单用户可借走全部流动性）
风险：单个用户可借走池中全部资金，导致其他存款人无法取款。
修复：引入单笔借款上限（如利用率不超过 95%）。

姓名：	日期：	得分：	/ 100
-----	-----	-----	-------